Ingegneria del Software — Soluzione del Tema 26/08/2022

Esercizio 1

Si consideri la seguente classe `SeriesStore` per memorizzare episodi di serie TV. Gli episodi sono rappresentati dalla classe immutabile `Episode` che espone tra gli altri un metodo per ottenere la serie di appartenenza di un episodio.

```
public class SeriesStore {
  // Ritorna il primo episodio della serie passata per parametro.
  // Ritorna null se la serie non esiste.
  public /*@ pure @*/ Episode getFirstEpisode(String series);

  // Ritorna l'episodio successivo a quello passato per parametro.
  // Ritorna null se tale episodio non esiste, ovvero se quello passato per parametro
  // e' l'ultimo della serie.
  public /*@ pure @*/ Episode getNextEpisode(Episode ep);

  // Ritorna il numero di episodi della serie successivi a quello passato per parametro.
  // Ritorna 0 se la serie non esiste.
  public /*@ pure @*/ int numNextEpisodes(Episode ep);

  // Aggiunge l'episodio in fondo alla sua serie di appartenenza. Se la serie ancora
  // non esiste, l'episodio viene aggiunto come il primo di quella serie.
  // Lancia una IllegalEpisodeException se l'episodio passato come parametro e' null.
  public void addEpisode(Episode ep) throws IllegalEpisodeException;
}

public /*@ pure @*/ Episode {
  // Ritorna la seria di appartenenza dell'episodio.
  public /*@ pure @*/ String getSeries();
... }
```

Domanda a)

Specificare in JML il metodo `numNextEpisodes`.

Soluzione

```
//@ requires ep != null
//@
//@ ensures \result == (getNextEpisode(ep) == null) ? 0 : 1+numNextEpisodes(getNextEpisode(ep))
public /*@ pure @*/ int numNextEpisodes(Episode ep);
```

Domanda b)

Specificare in JML il metodo `addEpisode`.

Soluzione

Definiamo `addEpisode` solo a partire da `getFirstEpisode` e `getNextEpisode` in quanto `numNextEpisodes` deriva da questi.

```
//@ ensures ep != null &&
//@
//@ (\forall String series; series != null && !series.equals(ep.getSeries());
//@    getFirstEpisode(series) == \old(getFirstEpisode(series)) ) &&
//@ (\forall Episode e; e != null && !e.getSeries().equals(ep.getSeries());
//@    getNextEpisode(e) == \old(getNextEpisode(e)) ) &&
//@
//@ getFirstEpisode(ep.getSeries()) == \old(getFirstEpisode(ep.getSeries())) == null ?
//@   ep : \old(getFirstEpisode(ep.getSeries()) &&
//@ getNextEpisode(ep) == null &&
//@ (\forall Episode e; e.getSeries().equals(ep.getSeries()) && !e.equals(ep);
//@   getNextEpisode(e) == (\old(getNextEpisode(e))) == null ?
//@     ep : \old(getNextEpisode(ep.getSeries())) )
//@
```

```
//@ signals (IllegalEpisodeException exc) ep == null &&
//@ (\forall Episode e; e != null; getNextEpisode(e) == \old(getNextEpisode(e))) &&
//@ (\forall String series; series != null;
//@   getFirstEpisode(series) == \old(getFirstEpisode(series)) )
public void addEpisode(Episode ep) throws WrongEpisodeException;
```

Domanda c)

Si consideri la seguente implementazione della classe `SeriesStore` che usa una mappa per associare a ogni serie la lista dei suoi episodi. Si ricorda che una Map<K, V> è una struttura dati che associa a chiavi di tipo K (`String` in questo caso) valori di tipo V (`List<Episode>` in questo caso). Una Map fornisce, fra gli altri, i metodi osservatori `get(K key)`, che restituisce il valore univoco associato alla chiave `key`, e `containsKey(K key)`, che restituisce `true` se esiste un valore associato alla chiave `key`. Altri metodi sono ad esempio `size()` e `isEmpty()`, di significato ovvio.

```
public class SeriesStore {
  private Map<String, List<Episode>> episodes;
... }
```

Per tale implementazione, si forniscano l'invariante di rappresentazione e la funzione di astrazione.

Soluzione

L'invariante di rappresentazione richiede che la mappa non sia nulla, non contenga chiavi nulle e non contenga liste di episodi nulle o vuote. Gli episodi in ogni lista non devono essere duplicati e la loro serie deve corrispondere a quella della chiave.

```
//@ private invariant
//@ episodes != null && !episodes.containsKey(null) &&
//@ (\forall String s; episodes.containsKey(s);
//@   episodes.get(s) != null && ! episodes.get(s).isEmpty() &&
//@   (\forall int i; i>=0 && i<episodes.get(s).size();
//@     episodes.get(s).get(i).getSeries().equals(s) &&
//@     !(\exists int j; j>=0 && j<i;
//@       episodes.get(s).get(i).equals(episodes.get(s).get(j)) ) ) )
```

La funzione di astrazione definisce `getFirstEpisode` e `getNextEpisode` in base al contenuto di `episodes`. Tutti gli altri metodi sono definiti a partire da questi.

```
//@ private invariant
//@ (\forall String s; s!=null; getFirstEpisode(s) ==
//@   episodes.containsKey(s) ? episodes.get(s).get(0) : null) &&
//@ (\forall Episode e; e!=null; getNextEpisode(e) ==
//@   (episodes.containsKey(e.getSeries()) &&
//@     (\exists int i; i>=0 && i<episodes.get(e.getSeries()).size()-1;
//@       episodes.get(e.getSeries()).get(i).equals(e) ) ) ?
//@   episodes.get(e.getSeries()).get(i+1) : null
```

Domanda d)

Si consideri una classe `SeriesStore2` che aggiunge un metodo `addEpisodes` che prende in ingresso una lista di episodi e li aggiunge allo store. Può tale classe essere definita come sottoclasse di `SeriesStore` in accordo con il principio di sostituzione?

Soluzione

La classe può essere definita come sottoclasse di `SeriesStore` perchè aggiunge un metodo senza alterare i metodi esistenti e senza violare alcun invariante pubblico di `SeriesStore`.

Esercizio 2

Si consideri la seguente classe Java:

```
public class SyncCouple {
  private int x, y;
  private String xlock = new String("xlock"), ylock = new String("ylock");
  public void setX(int x) {
    synchronized(xlock) { this.x=x; }
  }
  public void setY(int y) {
    synchronized(ylock) { this.y=y; }
  }
  public void setXYIfNotEqual(int i) {
    synchronized(xlock) {
      synchronized(ylock) {
        if(x!=y) { x=i; y=i; }
      }
    }
  }
  public boolean equalXY() {
    synchronized(ylock) {
      synchronized(xlock) {
        return x==y;
      }
    }
  }
}
```

Domanda a)

La classe è correttamente sincronizzata? In caso negativo spiegare, anche con un esempio, quale problema possa verificarsi e indicare come correggere il problema minimizzando i cambiamenti e senza stravolgere l'approccio alla sincronizzazione adottato.

Soluzione

La chiamata dei metodi `setXYIfNotEqual` e `equalXY` sullo stesso oggetto da parte di due thread diversi potrebbe portare ad un deadlock. Infatti il thread `T1`, che esegue il primo metodo, potrebbe acquisire il lock sull'attributo `xlock` per poi cedere il controllo al secondo thread `T2` che, eseguendo il metodo `equalXY`, acquisirebbe il lock su `ylock` per poi sospendersi in attesa di poter acquisire il lock su `xlock`, ma tale lock sarebbe in possesso di `T1` che si troverebbe a sua volta bloccato in attesa di acquisire il lock su `ylock` (in possesso di `T2`).

La soluzione più semplice al problema, che non modifica l'approccio alla sincronizzazione, è scambiare l'ordine di acquisizione dei lock in uno dei due metodi `setXYIfNotEqual` o `equalXY`, in modo da acquisire i lock nel medesimo ordine. Ad esempio modificando il metodo `equalXY` come segue:

```
  public boolean equalXY() {
    synchronized(xlock) {
      synchronized(ylock) {
        return x==y;
      }
    }
  }
```

Domanda b)

Si aggiunga alla classe `SyncCouple` un metodo `waitForZeroX()` che sospende il chiamante fino a quando l'attributo `x` non assume il valore zero.

Soluzione Il metodo va implementato come segue:

```
  public void waitForZeroX() throws InterruptedException {
    synchronized(xlock) {
      while(x!=0) xlock.wait();
    }
  }
```

Inoltre bisognerà aggiungere l'istruzione `xlock.notifyAll()` ai metodi `setX` e `setXYIfNotEqual` (all'interno del blocco sincronizzato su `xlock`).

Esercizio 3

```
1  public static void mistery(int x) {
2    if (x <= 1) {
3      return;
4    }
5    int m = x - 1;
6    while (x >= 0) {
7      x = x % m;
8      if (m > 1 || x >= 0) {
9        x = x -1;
10     }
11   }
12   return;
13 }
```

Si determini un insieme minimo di casi di test per ciascuno dei casi seguente di copertura.

1. Copertura delle istruzioni (statement coverage)
2. Copertura delle decisioni (edge coverage)
3. Copertura delle decisioni e delle condizioni (edge and condition coverage)

Soluzione

1. Copertura delle istruzioni (statement coverage). Essendoci due return, servono almeno due casi. Ad es. $x = 1$ (per uscire dal primo return) e $x = 2$ (per percorrere il while una volta) sono in realtà anche sufficienti.
2. Copertura delle decisioni (edge coverage). La copertura completa delle decisioni è impossibile, in quanto la condizione dell'if all'interno del ciclo è sempre verificata: x non può essere negativa, perché il corpo del ciclo è eseguito con $x >= 0$ e l'istruzione $x = x\%m$ non può rendere x negativo ($m > 0$). I casi precedenti $x = 1, x = 2$ sono sufficienti per massimizzare la copertura delle decisioni.
3. Copertura delle decisioni e delle condizioni (edge and condition coverage). Con i casi precedenti $x = 1, x = 2$ si copre la condizione $m > 1$ falsa; per rendere $m > 1$ vera, basta prendere $x = 3$.

Esercizio 4

Si considerino le seguenti classi Java:

```
abstract class Date {
  public abstract String dateDistance(Date d);
}
class DayDate extends Date {
  int day;
  public String dateDistance(Date p) {
    return "Distance of the two dates is" + dateDistance((DayDate) p);
  }
  public String dateDistance(DayDate p) {
    return ": " + (day-p.day) + " days";
  }
  public DayDate(int d) { day=d; }
}
class DayMonthDate extends DayDate {int month;
  DayMonthDate(int d, int m) { super(d); month = m; }
  public String dateDistance(Date p) {
    return "Distance in days and months is" + dateDistance((DayMonthDate) p);
  }
  public String dateDistance(DayMonthDate p) {
    return super.dateDistance(p) + ": " + (month - p.month) + " months";
  }
}
class DayMonthYearDate extends DayMonthDate {int year;
  DayMonthYearDate(int d, int m, int y) { super(d, m); year = y; }
  public String dateDistance(Date p) {
    return "Distance in d-m-y is" + dateDistance((DayMonthYearDate) p);
  }
  public String dateDistance(DayMonthYearDate p) {
    return super.dateDistance(p) + ": " + (year - p.year) + " years";
  }
}
```

e il seguente frammento di codice le cui righe sono state numerate per riferimento:

```
1  DayDate d1 = new DayDate(25);
2  DayDate d2 = new DayMonthDate(25, 2);
3  DayMonthDate d3 = new DayMonthDate(20, 4);
4  DayMonthYearDate d4 = new DayMonthDate(25, 2);
5  DayMonthYearDate d5 = new DayMonthYearDate(24, 3, 2022);
6  Date d6 = new DayDate(24);
7  System.out.println(d1.dateDistance(d2));
8  System.out.println(d1.dateDistance(d3));
9  System.out.println(d1.dateDistance(d6));
10 System.out.println(d2.dateDistance(d3));
11 System.out.println(d3.dateDistance(d2));
12 System.out.println(d3.dateDistance(d5));
13 System.out.println(d4.dateDistance(d5));
14 System.out.println(d5.dateDistance(d3));
15 System.out.println(d5.dateDistance(d6));
```

Domanda a)

Scrivere il numero di riga delle istruzioni (se ne esistono) che generano un errore in compilazione o un'eccezione a run time, chiarendo se il problema è a compile time o a run time, e spiegandone brevemente le ragioni.

Soluzione

La variabile `d4` è inizializzata incorrettamente, in quanto `DayMonthDate` non è sottoclasse di `DayMonthYearDate`. Sono quindi da eliminare anche le righe in cui `d4` viene riferita (compile time).

La chiamata `d5.dateDistance(d6)` genera a run time un ClassCastException, in quanto il cast del metodo `DayMonthYearDate.dateDistance(Date)` da Date a `DayMonthYearDate` non è possibile: il tipo dinamico di `d6` è infatti `DayDate`.

Sono quindi da eliminare le righe 4, 13 e 15.

Domanda b)

Dopo aver rimosso le linee identificate al punto precedente, scrivere cosa stampa il programma (indicando il numero di riga che produce ogni stampa).

Soluzione

7) : 0 days
8) : 5 days
9) Distance of the two dates is: 1 days
10) : 5 days
11) : -5 days
12) : -4 days: 1 months
14) : 4 days: -1 months